

Photobooth App — Implementation Plan

Canon EOS R50 · Windows · generated 05 September 2026

Context

You want a photobooth you run yourself at events: a **Canon EOS R50** tethered over USB-C, a guest-facing monitor showing live preview and results, and all camera control from software on your laptop. Today

`C:\Users\edwar\Projects\photobooth-app` is empty — this is greenfield.

Confirmed requirements:

Decision	Answer
Camera	Canon EOS R50 (APS-C mirrorless, 24.2MP, USB-C)
Operation	Attended (you drive the shutter), with a self-serve kiosk mode as a later option
Screens	Your laptop (operator) + one external monitor (guest-facing, fullscreen)
v1 outputs	Photo strip / collage template, printing, QR-code gallery download
Deferred	GIF/boomerang, green screen, cloud upload, SMS/email
Stack	Your call — recommendation below

The intended outcome: a single app you launch at an event, point at the R50, and run a full capture → composite → print/QR flow without touching the camera.

What the R50 specifically gives us

The R50 is a much better body for this than the older DSLRs, and it opens a path that removes the biggest risk from the project:

Path	Available on R50?	Verdict for a booth
EDSDK over USB-C	Yes — R50 is in the tethered-shooting supported list (TetherTools , works with EOS Utility / Lightroom / Capture One)	Primary. Wired, reliable, full-resolution capture.
UVC webcam mode over USB-C	Yes — the R50 is UVC/UAC compliant, 1080p30, no software at all	Dev environment + emergency fallback. Plug in, it's a webcam.
Clean HDMI out	Yes — configurable via the HDMI display setting in the movie menu (Canon manual)	Preview upgrade path. Genuinely viable, unlike older Rebels.
CCAPI	Yes, but Wi-Fi only and ships dormant, needing an activation tool from Canon's dev portal	Not primary — Wi-Fi at a venue is a liability.

Two consequences that shape the whole plan:

1. The USB port is mode-switched, so tethering and webcam mode are mutually exclusive. The R50 has a `Settings → USB Connection App` menu setting (`Photo Import/Remote Control` vs `webcam`). One cable, one mode at a time. So you cannot use UVC preview and EDSK capture simultaneously over USB.

2. But HDMI is a separate path, so preview and capture *can* be decoupled. Clean HDMI out → a cheap USB capture dongle → the browser reads it as a webcam at true 1080p, while USB-C independently runs EDSK for the shutter and the

full-res download. On the older DSLRs I originally planned for, this was blocked by the lack of clean HDMI. On the R50 it's a real option.

3. UVC mode means you can build the entire app before Canon approves your SDK access. This is the big one — see Step 0.

Recommended stack: .NET 10 backend + local web UI

One process: an ASP.NET Core app that owns the camera and serves a browser UI on localhost. You open two browser windows — operator controls on the laptop, fullscreen guest display dragged onto the monitor.

This is the answer to "desktop app or native app — both seem viable": it is both. The camera layer is native C# (what a native app buys you), the UI is web (what an Electron app buys you), and it ships as one self-contained `.exe` with no Node runtime and no bridge process.

Why this and not the alternatives:

- **Canon's EDSDK is a native C DLL** with a strict single-threaded + Windows-message-pump model. C#/.NET P/Invoke handles this cleanly. Calling it from Node (`node-ffi-napi`) or Python (`ctypes`) means marshalling callbacks across a runtime boundary that fights you on exactly the threading rules EDSDK cares about. This is the strongest technical constraint in the project.
- **Electron/Tauri** would still need that same C# process running alongside as a camera server — two runtimes to get a window. A browser in `--app` mode gives you the same fullscreen window for free.
- **libgphoto2** (what most open-source photobooths use: [photobooth-app](#), [PhotoboothProject/photobooth](#)) is not practical on Windows — those projects assume Linux/Raspberry Pi. Worth reading for UX ideas, not for the camera layer.

Concretely:

- **Backend:** .NET 10 (LTS), ASP.NET Core Minimal APIs + SignalR, **built for x64 explicitly** (EDSDK.dll is architecture-specific).
- **Frontend:** Vite + React + TypeScript, built into `wwwroot`. Two routes: `/operator` and `/display`.
- **Imaging:** SkiaSharp for compositing and print rendering (MIT-family license, no commercial-use question — unlike ImageSharp's split license).
- **QR:** QRCode (MIT).
- **Storage:** filesystem for photos + SQLite for session metadata.
- **Tests:** xUnit.

Step 0 — day one

Register for the Canon EOS Digital SDK. Free, but gated behind a developer registration and license agreement, and approval is not instant — it can take days. Apply at the Canon developer portal for your region ([Canon USA](#), [Canon Europe](#), [Canon Asia](#)). You'll need to state a use case.

Nothing in this plan blocks on that approval. Because the R50 does UVC natively, milestones M1–M3 and M5–M8 are built against a webcam-mode adapter that is a *real working photobooth* — just with 1080p stills instead of 24MP ones. EDSDK slots in at M4 and upgrades capture quality without touching the rest.

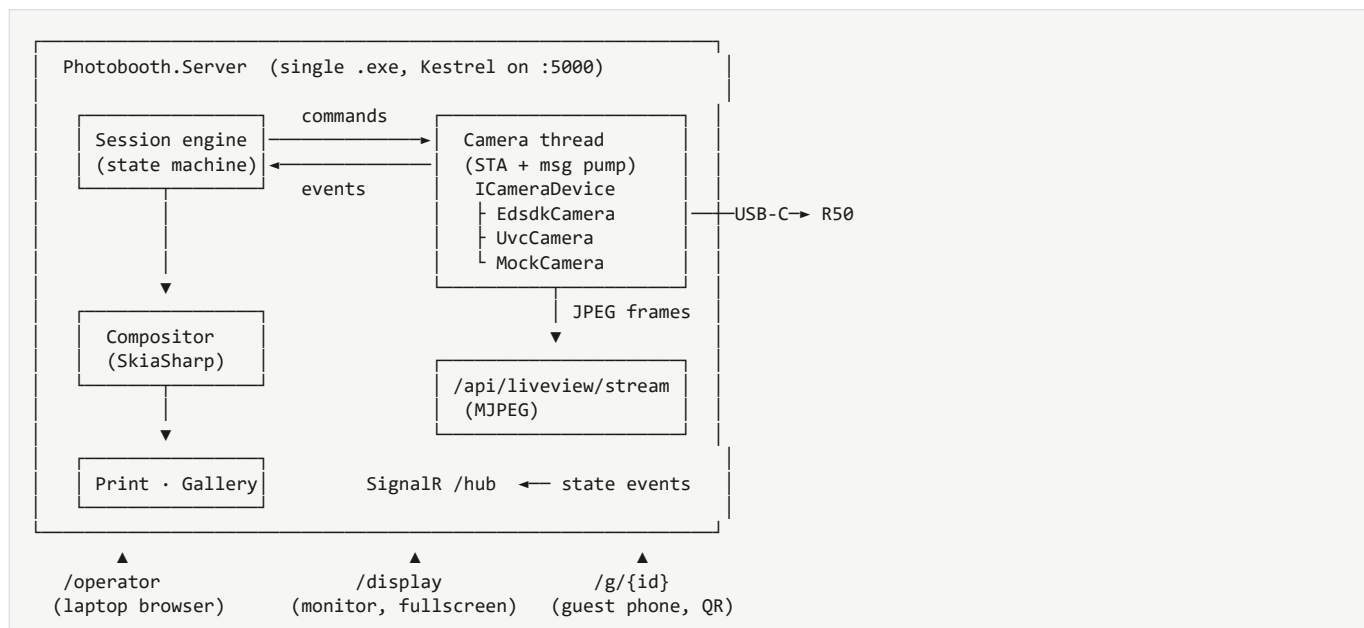
Hardware checklist while you wait:

- **DR-E18 / LP-E17 dummy battery with a USB-C PD adapter** (~\$20). Non-negotiable. The R50 will **not** power itself from the USB-C port while tethered — TetherTools explicitly notes no USB power on this body — so the dummy battery goes in the battery compartment and leaves the USB-C port free for data. A booth kills an LP-E17 in well under an hour.
- **A good, short USB-C cable.** The R50 is USB 2.0 (480 Mbit/s) despite the USB-C connector, so cable quality matters more than spec. Over ~15 ft you need an active extension.
- **Tripod**, fixed and taped-down camera position.
- *(Optional, for the crisp-preview upgrade)* micro-HDMI to HDMI cable + a USB capture dongle. Buy this only after M3 tells you whether EDSK live view is good enough.

R50 settings to lock in:

- **Settings** → **USB Connection App** → **Photo Import/Remote Control** (this is what puts the port in tether mode rather than webcam mode).
- **Airplane mode ON** under Communication Functions — TetherTools recommends this for tether stability, and it saves battery.
- Manual exposure, **manual white balance** (not AWB — guests' skin tone will shift shot to shot), fixed ISO.
- **JPEG only**, not RAW+JPEG. RAW roughly triples per-shot transfer time over a USB 2.0 link.
- Auto power off disabled, image review off.

Architecture



The camera thread is the critical design point. EDSK requires that all SDK calls happen on one dedicated thread that also runs a Windows message pump — camera events arrive as window messages. Get this wrong and you get intermittent hangs that are miserable to debug. So: one long-lived thread, a `Channel<CameraCommand>` in, an event stream out, and *nothing* else ever touches the SDK.

Preview source and capture source are deliberately separate concerns. The display page renders whatever the preview source gives it; the session engine asks the capture source for a photo. In EDSK mode both are the same device. In HDMI-dongle mode, preview is a browser `getUserMedia` stream while capture is EDSK. Swapping between them is configuration, not a rewrite.

The camera abstraction

```
interface ICameraDevice {
    Task ConnectAsync();
    IAsyncEnumerable<LiveViewFrame> LiveViewAsync(Cancellation token ct);
    Task<CapturedPhoto> CaptureAsync(); // returns path to downloaded JPEG
    Task<CameraSettings> GetSettingsAsync();
    Task ApplySettingsAsync(CameraSettings s); // ISO, aperture, shutter, WB
    event EventHandler<CameraEvent> Events; // disconnect, error, busy
}
```

Three implementations, in build order:

- 1 **MockCamera** — replays sample JPEGs, synthesizes live-view frames with a moving element and timestamp so you can see the stream is live. Lets you run the full app with nothing plugged in, and makes the session tests deterministic.
- 2 **UvcCamera** — the R50 in webcam mode. Preview is `getUserMedia` in the browser; "capture" grabs a full-resolution still frame from the video track. Needs **no SDK, no registration, no extra software**. This produces a genuinely working photobooth at 1080p, and doubles as your emergency fallback if EDSDK misbehaves at a venue.
- 3 **EdsdkCamera** — the real target. Direct P/Invoke against the current EDSDK for 24MP capture and camera settings control.

On (3): the .NET wrappers on GitHub ([thrixton/EDSDK.NET](https://github.com/thrixton/EDSDK.NET), [esskar/Canon.Eos.Framework](https://github.com/esskar/Canon.Eos.Framework)) target much older SDK versions (3.x era, long before the R50 existed) and none is actively maintained on NuGet. Plan to write our own thin P/Invoke layer against the headers in the SDK download — a few hundred lines for the subset we need — using those repos as reference for the awkward parts (property IDs, EVF stream setup, the object-transfer event dance).

I originally planned a [digiCamControl](#) HTTP adapter as the no-registration fallback. **Dropping it for the R50:** `digiCamControl` drives Canon bodies through its own bundled EDSDK wrapper, and a body as recent as the R50 may not be recognised by its Canon support. Unverified, and the R50's native UVC mode is a strictly better fallback anyway — no second application to babysit at an event.

Project layout

```
photobooth-app/
├── src/
│   ├── Photobooth.Core/      Session state machine, templates, domain. No I/O.
│   ├── Photobooth.Cameras/   ICameraDevice, camera thread, Mock/Uvc/Edsdk adapters
│   ├── Photobooth.Imaging/   SkiaSharp compositor, print renderer, QR
│   ├── Photobooth.Server/    ASP.NET Core: APIs, SignalR hub, gallery, wwwroot
│   └── Photobooth.Web/       Vite + React + TS → builds into Server/wwwroot
├── tests/
│   ├── Photobooth.Core.Tests/
│   ├── Photobooth.Imaging.Tests/    golden-image comparisons
│   └── Photobooth.Cameras.Tests/    contract tests, run against MockCamera
├── templates/                  strip/collage layout definitions + PNG overlays
├── data/                      sessions.db, captures/, output/ (gitignored)
└── docs/
```

Photobooth.Core depends on neither ASP.NET, SkiaSharp, nor the SDK — the session logic is testable in isolation, which matters because the state machine is where the subtle bugs live (double-fire on shutter, retake mid-countdown, disconnect during a sequence).

Milestones

Each milestone ends in something you can run and look at.

M1 — Skeleton + mock camera end to end

Solution, projects, DI wiring. `MockCamera` behind `ICameraDevice`. Kestrel serving a bare `/operator` page with a Capture button and `/display` showing the mock stream. **Verify:** `dotnet run`, open both pages, click Capture, see a JPEG land in `data/captures/` and appear on the display page.

M2 — Session state machine + two-window flow

Formalize the flow: `Idle` → `Arming` → `Countdown(n)` → `Capturing(shot k of N)` → `Reviewing` → `Composing` → `Done`. SignalR pushes state to both windows. Operator gets Start / Retake last / Abort / Accept. Display gets the countdown overlay, a flash effect, and per-shot review. **Verify:** unit tests over the state machine (illegal transitions rejected, retake rewinds exactly one shot, abort from any state returns to `Idle`). Run a 3-shot session on the mock and confirm both windows stay in sync.

M3 — Real R50, webcam mode (`UvcCamera`)

Set the camera to webcam mode, run the whole flow against it. Preview via `getUserMedia` on the display page; capture pulls a still from the video track. **Verify:** a complete 4-shot session against the actual R50 with **no SDK access required**. At this point you have a working photobooth. If Canon approval is slow, everything downstream still proceeds.

M4 — Real R50, tethered (`EdsdkCamera`)

Switch `USB Connection App` to `Photo Import/Remote Control`. Camera thread with message pump. Connect, session open, live view start/stop, EVF download loop, `EdsSendCommand(TakePicture)`, the `ObjectEvent` → `DirItemRequestTransfer` download callback, read/write ISO/Av/Tv/WB. Handle disconnect and reconnect without restarting the app. **Verify:** hardware checklist — connects on app start; live view within 2s; **10 consecutive captures with no hang**; unplug USB mid-session and confirm a clean error state rather than a crash; replug and reconnect. Measure shutter-press → file-on-disk latency (target under ~2s for a 24MP JPEG over USB 2.0).

Decision point: compare EDSDK live-view quality on the monitor against what you saw in M3. If the EVF stream is too soft at your monitor size, buy the HDMI dongle and wire preview to the capture-card stream while EDSDK keeps the shutter. The abstraction already supports this; it's a config change plus ~30 lines.

M5 — Templates and compositing

A template is JSON: output size in pixels + DPI, background/overlay PNG, and N photo slots each with position, size, rotation, and fit mode. Ship two: classic 2×6 strip (4 shots) and 4×6 2×2 grid. Compositor loads captures, applies cover-fit crop, draws slots, draws overlay, writes the final PNG. **Verify:** golden-image tests — composite known input JPEGs against each template and diff against a checked-in expected PNG. Visually confirm a real session's output at 300 DPI.

M6 — Gallery + QR

SQLite session records. `/g/{sessionId}` serves a mobile-friendly page with the composite and the individual shots, each downloadable. QR encodes `http://<LAN-IP>:5000/g/{id}` and shows on the display at session end. Auto-detect the LAN IP; make it overridable in settings. **Verify:** run a session, scan the QR with a phone on the same Wi-Fi, download the composite. Confirm the QR still resolves after an app restart.

Known constraint: the QR only works if guests are on the same network as the laptop. At a venue with no usable Wi-Fi you need a travel router or a phone hotspot that both the booth and guests join. Cloud upload would remove this and is deliberately out of scope for v1 — the gallery sits behind an interface so it can be added without touching the session flow.

M7 — Printing

Render the composite at the target physical size (4×6 at 300 DPI = 1200×1800 px) and send to a Windows printer via `System.Drawing.Printing`, with paper size, orientation, and borderless margins configurable per template. **Print through a background queue**, never inline in the session flow — the next guests should not wait on a printer. Show queue state on the operator screen. **Verify:** print to whatever you have now (or Microsoft Print to PDF) and *measure* the output — a 4×6 must come out actually 4×6, not scaled-to-fit. Queue three prints and confirm sessions stay responsive. Kill the printer mid-queue and confirm the app surfaces the error without losing photos.

M8 — Packaging and event hardening

Single self-contained `x64` publish. A launcher that starts the server and opens both windows (`msedge --app=http://localhost:5000/display` on the monitor, `/operator` on the laptop). Settings screen for camera mode, template, countdown length, shot count, printer, output folder. Crash-safe: sessions persist, and photos hit disk before any compositing. **Verify: a full dress rehearsal** — publish, run on a clean boot, do 20 sessions back to back, print each, scan a QR. Watch for memory climbing across sessions (the live-view loop is the likely leak) and for the R50 warming up under sustained live view — it's a compact body and this is worth measuring rather than assuming. This rehearsal is the real acceptance test.

M9 (later) — Self-serve kiosk mode

Attract/idle screen, big touch or hardware-button start, auto-advance with no operator input, operator view demoted to a hidden admin panel. Cheapest trigger: a USB footswitch or wireless presenter that emits a keystroke — no extra integration needed.

Risks

Risk	Mitigation
EDSDK registration delayed or denied	<code>UvcCamera</code> (M3) is a complete working booth with no SDK at all
EDSDK threading bugs (hangs)	One dedicated thread + message pump + command queue, nothing else touches the SDK. Established in M4 before features pile on.
EDSDK live view too soft on the monitor	R50 has clean HDMI — dongle upgrade confirmed viable, and preview/capture are already separate concerns
R50 battery dies mid-event	DR-E18 dummy battery; USB-C cannot power this body while tethered
USB tether dropouts	Short quality cable, airplane mode on, explicit reconnect path tested in M4, UVC fallback available
Venue has no usable Wi-Fi for QR	Travel router or hotspot; documented as an event prerequisite
Printer is the flow bottleneck	Background print queue from the start (M7)

First actions on approval

- 1 Submit the Canon EDSK developer registration — blocks nothing, but the clock starts.
- 2 Order the DR-E18 dummy battery and a short quality USB-C cable.
- 3 `git init` and scaffold the solution per the layout above.
- 4 Build M1 — mock camera, two pages, one capture round-trip.